

Using the Eigenfaces approach and Machine Learning Classifiers, facial Emotion Recognition through the FEC2013 database

Tristan Jones

Abstract:

Facial emotion recognition (FER) is a critical domain of computer vision with numerous applications in human-computer interactions, mental health diagnostics, and emotion recognition [1,2]. This project looks to utilize the FER-2013 dataset, which was created for the ICML 2013 Workshop Challenges in Representation Learning [3,4]. This workshop focused on three challenges: the blackbox learning challenge, the facial expression recognition challenge, and the multimodal learning challenge [3,4]. This dataset comprises about 35,000 different 48x48 pictures obtained using the Google search API, then labeled and formatted [4]. This study aims to adapt the eigenfaces algorithm for Principal Component Analysis (PCA) [5], using this primarily for dimensionality reduction. Then, from there, this study aims to employ and compare a few different machine learning algorithms for classification to enhance the efficiency and accuracy of the emotion recognition algorithm. Here, our methodology has four primary stages: data loading and preprocessing, eigenfaces dimensionality reduction, training, and model evaluation.

The FER2013 dataset has been used extensively in various machine learning implementations. During the ICML 2013 workshop, more than 50 teams submitted models with varying success, even up to 71% accuracy in prediction [4]. At the same time, another study that utilized the database and used deep learning techniques boasted a 75% accuracy on FER2013 [6]. These implementations were highly successful and showcased this dataset's usefulness for training models for facial emotion recognition. This project's results demonstrate

the efficiency of combining PCA with machine learning techniques to achieve robust emotional classification.

Background:

As noted, the FER 2013 dataset was curated using the Google image search API and consists of about 35,000 48x48 pixel images [4]. These images are labeled into one of seven emotion label categories: Angry, Disgust, Fear, Happy, Neutral, Sad, and Surprised. While this dataset is well known and useful for machine learning models, it remains a challenging benchmark due to certain emotions being underrepresented vs other emotions [4]. Given the seven different classifications of emotions, we have a baseline chance of randomly choosing the correct emotion at about 14%; however, when tested, human subjects only had an accuracy of $68 \pm 5\%$. As mentioned, multiple implementations boast an accuracy greater than 70% [4,6].

Methods:

This project is essentially divided into four different stages:

1. Data Loading and Preprocessing
2. Dimensionality reduction using PCA/Eigenfaces
3. Model Training
4. Model Evaluation

Data Loading and Preprocessing:

The FER 2013 dataset was loaded using a custom function to scale through the various file layers and load each image into the dataset. The pixel values themselves were normalized by dividing by 255 to make training with the data easier in the future. The photos were already in grayscale and 48x48, so they just needed to be loaded and then put into the data matrix to be used for the eigenfaces approach.

Dimensionality Reduction with PCA (Eigenfaces):

The data was very high-dimensional, so the idea was to utilize PCA and the eigenfaces approach to get the most valuable data. At first, I tried using the large covariance matrix, but this ended up using too much system memory, so I opted to utilize the small covariance matrix instead. The difference being that the large covariance matrix was somewhere on the order of 10,000x10,000 values vs the ~2000x2000 values of the small covariance matrix. This was significantly easier to utilize and takes up much less memory. From there, I calculated the principal components that explained more than 95% of the variance, which happened to be about 257 principal components. I utilized that for the next steps during the training.

Classifier Training:

With the data cleaned, we now have the principal components that explain a greater degree of the variance. I then utilized the SKLearn toolkits to implement several different models, including a K-Nearest Neighbor approach (KNN), a Shallow Neural Network(SNN), and a Random Forest (RF). The K-Nearest Neighbor approach was used previously in assignment 3 to evaluate the eigenfaces approach, where K was one, so I implemented it that way, and additionally, I wanted to implement a method that looked at K=3 as a comparison to the K=1 approach. The SNN has a single hidden layer with 100 neurons, ReLU activation, and uses the Adam optimizer. The Adam optimizer speeds up gradient descent. I did a grid search at one point and tested several different variations of the SNN, and this one came out on top. Lastly, I utilized a Random Forest approach with 100 trees. Again, I used a grid search here, and this amount worked the best for what we were trying to accomplish.

Model Evaluation:

Models were assessed based on accuracy, runtime, cross-validation performance, and inference speed. 5-fold cross-validation was performed to evaluate model stability. We wanted to ensure that it wouldn't drastically affect the model's outcome even if we chose a random subset of the data. Additionally, inference time for each classifier was recorded to compare the efficiency of utilizing each model for predictions. Visualization plots were generated to present accuracy, runtime, cross-validation stability, and inference time comparisons.

Prerequisites:

- Language: Python
- Libraries: NumPy, scikit-learn, Matplotlib, PIL
- Memory usage: 1980.16 MB

- CPU Usage: 4 cores

To run this program, you must download and unzip the folder. Then you can change the path to the “archive” folder on line 39 to whatever your path is on your device. From there, you should be able to run the code. It will print out multiple statements to tell you what it is currently working on, so you can follow along in the terminal. At the end, it will create the plots seen below.

Visualizations:

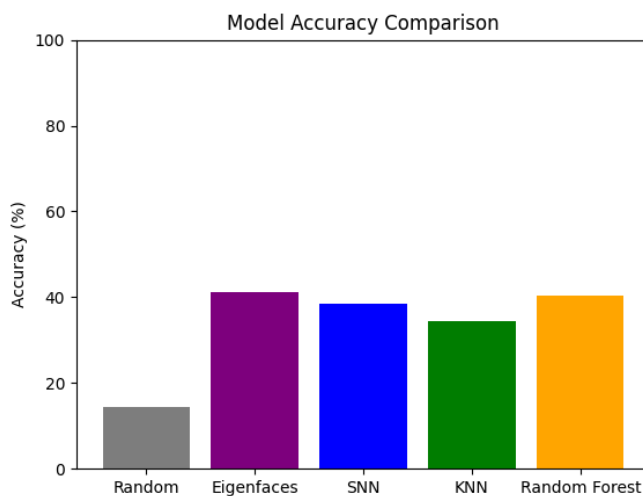


Figure 1: Model Accuracy Comparison

This bar chart shows the accuracy of all of the models compared to the baseline, which is a random choice, and between the Eigenfaces 1-NN vs the Shallow NN vs. 3-NN and the Random Forest approach. As we can see here, the eigenfaces approach is still very high even after training. However, other methods could get better if we had sufficient data.

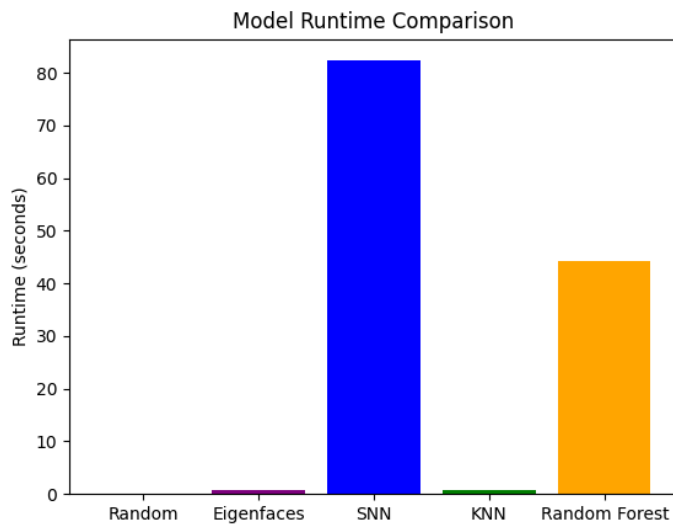


Figure 2: Model Runtime Comparison

This plot looks at the training time needed for the different types of models. No training is required for any K-Nearest Neighbor approaches, so it's just a calculation right away. However, with the Neural Network and Random Forest approaches, they need to train the model before predicting the facial emotion.

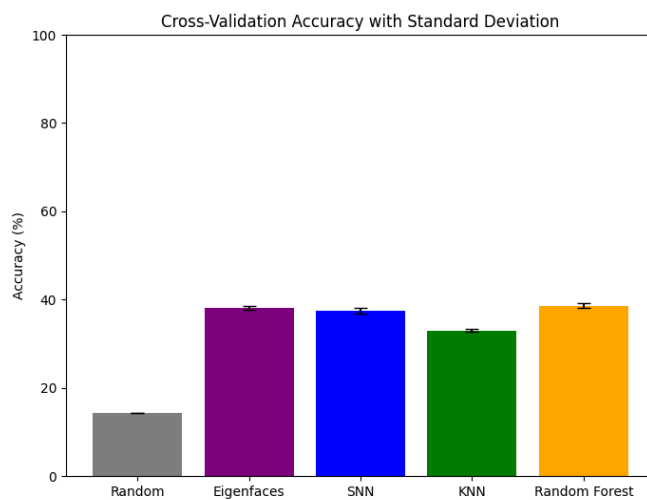


Figure 3: Cross-Validation Accuracy with Stability

This figure looks at the cross-validation accuracy. In this test, we are running cross-validation 5 times. This essentially trains the model on different subsets of the training data and looks at how well it can predict the outcome.

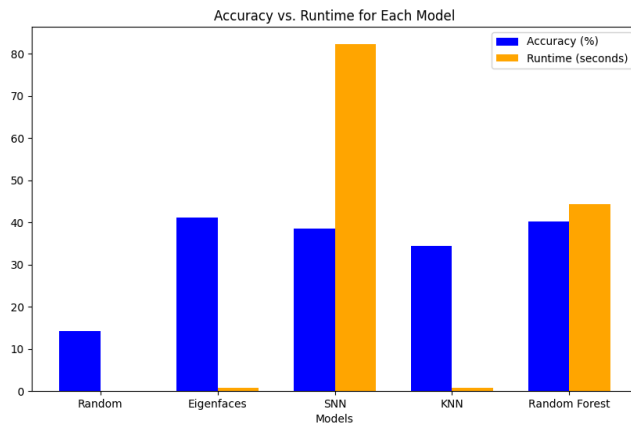


Figure 4: Accuracy vs. Runtime

This shows both the accuracy and runtime together in one plot. From this, we can see that the random forest model may be the best of both worlds here, given that it has a higher accuracy, but the runtime is still not too long.

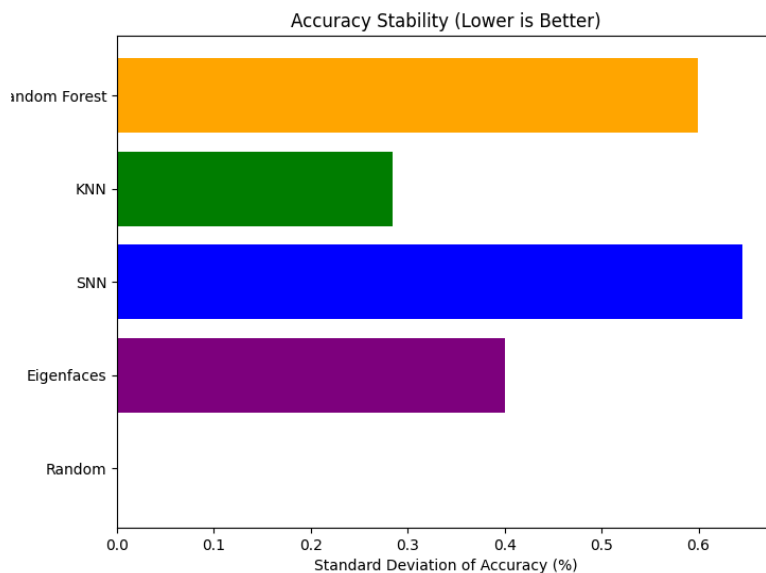


Figure 5: Accuracy Stability

This is also from the Cross Validation, similar to Figure 3, where these are just the standard deviation values. We can tell from this information whether there will be dramatic changes to the outcome given that we change the training data. All of these are pretty low, so there shouldn't be significant changes between them.

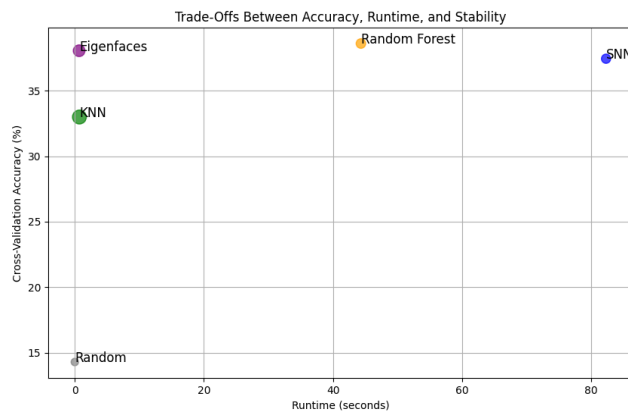


Figure 6: Trade-Offs Between Accuracy, Training Time, and Stability

This shows the trade-offs between the training time and accuracy. From this, we can see that all models have very similar accuracies; however, the SNN and RF approaches will have longer implementations due to training time.

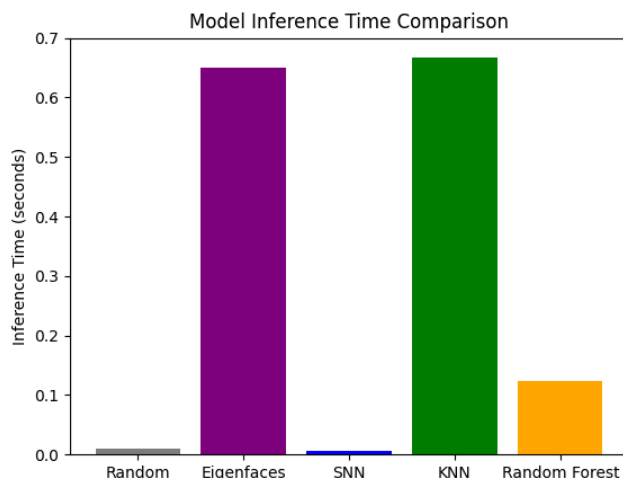


Figure 7: Model Inference Time Comparison

This figure looks at the time it takes to run the predictions after the models have been trained. This is important because the Shallow Neural Network and Random Forest approaches are significantly faster than the K-Nearest Neighbor approaches. The advantage of training an ML model is that it can evaluate any inputs substantially faster than if you need to calculate it manually.

Conclusions:

In conclusion, this project effectively demonstrates the capability for utilizing the eigenfaces method for dimensionality reduction in conjunction with other machine learning classifiers. Overall, the Shallow Neural Network achieved the best results, regardless of the high training time. This is to some extent the expected outcome. The Neural Network is very good at classifying data, and forward propagation is rapid after training is completed. The Random Forest was the most middling in every aspect, being that it was the middle of the pack in both training and accuracy. These tests highlight the effectiveness of combining PCA with ML algorithms for handling high-dimensional data efficiently, because I was able to run all of this with only 2GB of memory. While this has been shown to be better than a random choice, it's still not the most optimal compared to previously established results. Future work on this project would likely be concerning pushing the limits of this further to see what other things I could tweak to increase the success rate.

Resources:

- [1] Zeng, Z., Pantic, M., Roisman, G. I., & Huang, T. S. (2009). "A Survey of Affect Recognition Methods: Audio, Visual, and Spontaneous Expressions." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(1), pp. 39-58.
- [2] Calvo, R. A., & D'Mello, S. (2010). "Affect Detection: An Interdisciplinary Review of Models, Methods, and Their Applications." *IEEE Transactions on Affective Computing*, 1(1), pp. 18-37.
- [3] Goodfellow, I., Erhan, D., Carrier, P., et al. (2013). "Challenges in Representation Learning: A Report on Three Machine Learning Contests." *Neural Networks*, 64, pp. 59-63.
- [4] I. Goodfellow, D. Erhan, P. Luc Carrier, A. Courville, and Y. Bengio, "Challenges in Representation Learning: A Report on Three Machine Learning Contests," *Neural Networks*, vol. 64, pp. 59–63, 2015.
- [5] Turk, M., & Pentland, A. (1991). "Eigenfaces for Recognition." *Journal of Cognitive Neuroscience*, 3(1), pp. 71-86.
- [6] Khanzada, S., Chughtai, M. W., & Shaikh, A. R. (2020). "Hybrid Deep and Handcrafted Feature-Based Facial Expression Recognition Using FER2013." *arXiv preprint arXiv:2004.11823*.
- [7] M. Sambare, "Facial Expression Recognition (FER-2013) Dataset," *Kaggle*, 2021. [Online]. Available: <https://www.kaggle.com/datasets/msambare/fer2013>
- [8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <http://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>
- [9] GitHub, "GitHub Copilot: Your AI Pair Programmer," *GitHub*, 2024. [Online]. Available: <https://github.com/features/copilot>